

PRD: AI for Industrial Knowledge Intelligence — Unified Asset & Operations Brain

ET AI Hackathon 2026 — Problem Statement 8

Theme: Industrial Intelligence / Document Management / Knowledge Engineering / Quality

0. Source-of-Truth Note

This PRD is built strictly against the actual PDF text (confirmed by screenshot) for Problem Statement 8. Three things from the original request's template do **not** appear in the real problem statement and are therefore treated as **assumptions/enhancements**, not requirements:

- The PDF does not mandate a specific tech stack (Claude API/ChromaDB/FastAPI/React) — that is *your* implementation choice, compatible with the brief's "Suggested Technologies."
 - The PDF's five "what you may build" agents are explicitly **illustrative, not mandatory** ("These examples are illustrative only").
 - The PDF requires a GitHub repo, architecture diagram, presentation deck, and demo video as deliverables — not a Product Requirements Document itself. This PRD is the internal planning artifact your team uses to build those four deliverables.
-

1. Executive Summary

What it is: A RAG-powered Industrial Knowledge Intelligence platform that ingests heterogeneous plant documents (manuals, SOPs, P&IDs, work orders, incident reports, inspection records) and a knowledge graph of assets/equipment/failures, and exposes both through a cited, conversational AI Copilot plus an Asset 360 view.

Who it's for: Plant operators, maintenance/reliability engineers, field technicians, quality engineers, operations managers, and knowledge/enterprise admins at asset-intensive industrial sites (manufacturing, energy, process industries).

Problem it solves: Per the source material, professionals in asset-intensive industries lose <cite index="0-1">an average of 35% of their working hours searching for information, clarifying instructions, or recreating documents that already exist somewhere in the organisation</cite>. Indian plants specifically run <cite index="0-1">7

to 12 disconnected document systems

, and this fragmentation is estimated to drive 18–22% of unplanned downtime. A retirement wave threatens to erase undocumented tribal knowledge permanently.

Why existing search fails: Enterprise/document search returns files, not answers — it has no concept of *equipment*, *failure mode*, or *maintenance history* as first-class entities, no cross-document reasoning, no citation-grounded synthesis, and no way to detect that a procedure is missing or outdated rather than simply "not indexed."

Core value proposition: One conversational entry point that answers operational questions the way a 20-year veteran engineer would — citing the exact manual page, connecting a current fault to past incidents, and surfacing the right SOP — regardless of which of the 7–12 original systems the answer lives in.

Expected impact: Reduced mean-time-to-answer for operational queries (target: minutes → seconds), reduced repeat/unplanned downtime attributable to missing failure-history context, faster onboarding for new engineers, and audit-ready compliance evidence generation.

2. Problem Statement Analysis (Extracted, Not Invented)

Exact business problem: Knowledge fragmentation in industrial operations is not a file management problem. It is a safety problem, a quality problem, and an operational efficiency problem — and it compounds over time.

Root causes (from source):

- Documents scattered across 7 to 12 disconnected document systems — P&IDs/drawings in one, work orders in another, SOPs in a third, inspection records in a fourth, regulatory submissions in email archives.
- An impending **knowledge cliff**: an estimated 25% of India's experienced industrial engineers and operators will retire within the next decade, taking undocumented operational knowledge with them, permanently.

Current workflow (implied): An engineer facing an anomaly manually checks multiple systems/binders/emails, asks colleagues, or recreates a document from memory — the 35%-of-time sink described above.

Current pain points: No single place to ask a question; no linkage between a live fault and historical failure/incident data; regulatory and quality evidence assembled manually; knowledge never captured before an expert leaves.

Knowledge fragmentation issues: Same equipment's manual, maintenance history, SOP, and incident record live in different, unlinked systems with no shared identifier (e.g., equipment tag) connecting them.

Operational risks: Decisions made <cite index="0-1">without complete equipment history or failure pattern context</cite>, contributing directly to unplanned downtime; safety risk from outdated/conflicting procedures; compliance risk from scattered regulatory submissions.

Why traditional search fails: Keyword/file search returns documents ranked by term match, not by asset relevance, recency, or cross-document synthesis — it cannot answer "what caused this before" or "is this procedure current."

Why industrial context matters: An answer is only useful if grounded to the exact asset, the exact revision of a manual, and traceable evidence — hallucinated or uncited answers in a plant setting are a safety liability, not just an inconvenience.

Key constraints from the original problem statement:

- Must ingest **heterogeneous** documents: <cite index="0-1">engineering drawings, maintenance records, safety procedures, inspection reports, operating instructions, project files</cite> — <cite index="0-1">across structured and unstructured formats</cite>.
- Must be <cite index="0-1">queryable, actionable, and continuously updated at the point of need, across any device or function</cite> — i.e., **mobile-first**, not desktop-only.
- Suggested (not mandated) technology directions: <cite index="0-2">RAG over heterogeneous industrial document corpora, Knowledge Graphs & Industrial Ontology Engineering, Computer Vision (P&ID parsing, drawing digitisation), OCR & Document Intelligence, QMS integration, and Agentic AI for maintenance and compliance workflows</cite>.
- Expected deliverables: <cite index="0-2">Working Prototype, Architecture Diagram, Presentation Deck, Demo Video</cite>.
- Evaluation focus: <cite index="0-2">entity extraction accuracy across document types, query answer quality on domain-expert benchmark questions, knowledge graph linkage completeness, time-to-answer versus traditional search, compliance gap detection accuracy, and demonstrated improvement in cross-functional knowledge discovery</cite> — ideally validated with real industrial document samples.
- Judging weights: <cite index="0-2">Innovation 25%, Business Impact 25%, Technical Excellence 20%, Scalability 15%, User Experience 15%</cite>.

3. Product Vision

Vision statement: Every person on the plant floor gets a cited, trustworthy answer to any operational question in seconds, from any device.

Mission: Unify a plant's fragmented documents and tribal knowledge into one living, queryable "operations brain" before it's lost.

Long-term vision: An always-current digital twin of institutional knowledge — auto-updating as new work orders, incidents, and manuals arrive — that proactively flags gaps and risks before they cause downtime.

Hackathon MVP vision: Prove the core loop end-to-end on a realistic synthetic dataset: ingest → extract entities/graph → ask a hard cross-document question → get a cited, correct answer faster than manual search, plus one "wow" feature (similar-incident discovery or knowledge-gap detection).

Product principles:

1. Never answer without a citation.
 2. Assets, not documents, are the organizing unit.
 3. Mobile/field-usable by default, not a desktop afterthought.
 4. Say "I don't know" over hallucinating in a safety-relevant domain.
 5. Every gap the system finds is as valuable as every answer it gives.
-

4. Target Users and Personas

Plant Operator — Goals: quick answers during shift without leaving the control room. Pain points: manuals scattered, tribal knowledge with senior operators. Typical Q: "What's the startup sequence for Pump P-101?" Current workflow: call a supervisor or dig through a binder. Value: instant, cited procedure lookup on a tablet/HMI screen.

Maintenance Engineer — Goals: diagnose faults fast, minimize downtime. Pain points: no linkage between symptom and past failure history. Typical Q: "Show past failures similar to this vibration pattern on C-204." Value: similar-incident discovery + Asset 360.

Reliability Engineer — Goals: reduce recurring failures, run RCA. Pain points: incident data siloed, patterns invisible across sites. Typical Q: "What recurring failure modes exist for this equipment class?" Value: pattern detection across the knowledge graph.

Field Technician — Goals: get the right SOP on-site, hands-free/mobile. Pain points: poor signal/desktop-only tools, PDF manuals hard to search on phone. Typical Q: "Which SOP applies to this pressure anomaly?" Value: mobile-first AI Copilot with photo/OCR capture.

Quality Engineer — Goals: ensure procedures match regulatory requirements. Pain points: manual audits, conflicting document versions. Typical Q: "Is this SOP compliant with the latest OISD norm?" Value: compliance-gap detection, evidence packages.

Operations Manager — Goals: reduce unplanned downtime, see plant-wide knowledge health. Pain points: no visibility into where knowledge gaps exist. Typical Q: "Which

assets have the most unresolved incidents?" Value: analytics dashboard.

Knowledge Administrator — Goals: keep the corpus current and correctly ingested. Pain points: manual re-uploading, no visibility into failed parses. Typical Q: "Which documents failed OCR this week?" Value: ingestion monitoring, admin panel.

Enterprise Admin — Goals: security, access control, auditability. Pain points: no RBAC/audit trail across siloed systems. Value: org isolation, audit logs, role management.

5. Jobs To Be Done

- When I hear an unfamiliar alarm on the floor, I want to ask what it means in plain language, so I can respond correctly without waiting for a supervisor.
 - When equipment fails, I want to see every past failure of that exact asset, so I can rule out repeat causes fast.
 - When I'm about to be audited, I want evidence that current procedures match regulations, so I can pass compliance review without weeks of manual collation.
 - When a senior engineer retires, I want their undocumented know-how captured beforehand, so the plant doesn't lose it permanently.
 - When I get an AI-generated answer, I want to see exactly which document and page it came from, so I can trust it enough to act on it.
-

6. User Stories (Grouped by Persona)

Plant Operator

- As a plant operator, I want to ask "what's the shutdown procedure for Pump P-101" in natural language, so that I get the exact SOP step-by-step without searching manually.
 - AC: Answer returns within 5s, cites SOP document + page, matches the real procedure text.

Maintenance Engineer

- As a maintenance engineer, I want to see all past work orders and incidents for an asset in one view, so that I can diagnose faster.
 - AC: Asset 360 page loads maintenance history, incidents, and manuals for a given equipment tag in one screen.

Reliability Engineer

- As a reliability engineer, I want the system to flag recurring failure patterns automatically, so that I can prioritize RCA investigations.
 - AC: Dashboard lists top 5 recurring failure modes ranked by frequency over a selectable time window.

Field Technician

- As a field technician, I want to use the Copilot on my phone with poor connectivity, so that I can get answers on-site.
 - AC: Mobile-responsive chat UI works on 3G-equivalent throttled connection, response degrades gracefully (cached/offline notice) rather than failing silently.

Quality Engineer

- As a quality engineer, I want the system to flag SOPs that haven't been reviewed in over 2 years, so that I can prioritize updates.
 - AC: Knowledge-gap report lists documents by staleness (last-modified date) above a configurable threshold.

Operations Manager

- As an operations manager, I want a dashboard of most-searched assets and unresolved questions, so that I can direct documentation effort where it matters.

Knowledge Administrator

- As a knowledge admin, I want to see failed ingestion jobs with error reasons, so that I can fix and re-run them.
 - AC: Admin panel lists failed jobs with document name, failure stage (parse/OCR/embed), and retry button.

Enterprise Admin

- As an enterprise admin, I want role-based access control per document/asset, so that sensitive data isn't exposed to unauthorized users.

7. Complete Feature Requirements

7.1 Multi-Source Knowledge Ingestion

Supported formats (hackathon-realistic subset in bold; rest as stretch): PDF manuals, SOPs (PDF/DOCX), CSV/Excel maintenance & work-order exports, scanned/image documents (OCR); stretch: live DB/API connectors, structured drawing metadata.

- **Upload flow:** drag-drop or mobile camera capture → client-side type/size validation → upload to object storage → job enqueued → status polled/pushed to UI.

- **Batch ingestion:** a "folder upload" mode accepts a zip or multi-file selection; each file becomes its own ingestion job.
- **Connectors (P2/stretch):** generic REST/CSV pull connector stub – out of scope for hackathon beyond a mocked example.
- **Validation:** file type allowlist, max size (e.g., 25MB), virus/malware scan stub, duplicate-hash check before processing.
- **Parsing:** format-specific extractors (PDF text layer, DOCX via python-docx, XLSX via openpyxl/pandas, images via OCR).
- **OCR:** Tesseract (or PaddleOCR) applied to scanned pages/images; confidence score stored per page.
- **Metadata extraction:** filename, upload date, document type (classified), detected equipment tags, page count.
- **Deduplication:** SHA-256 content hash; if matched, link to existing document instead of re-ingesting.
- **Versioning:** new upload with same detected title/asset supersedes prior version; old version retained but flagged "superseded."
- **Reprocessing:** admin-triggered re-run of parse/OCR/embedding for a document (e.g., after a pipeline bug fix).

7.2 Industrial Knowledge Processing Pipeline (step-by-step)

1. **Ingest** raw file → object storage.
2. **Parse:** extract raw text + layout (page/section boundaries); route scanned pages to OCR.
3. **Table extraction:** detect and extract tabular regions (e.g., maintenance schedules) into structured rows, not flattened text.
4. **Layout-aware extraction:** preserve heading hierarchy so a "Section 4.2 Shutdown Procedure" stays attributable.
5. **Chunking:** semantic chunking within layout boundaries (see Section 17).
6. **Metadata enrichment:** attach document type, source, page number, section title to every chunk.
7. **Asset/equipment tag extraction:** regex + NER pass to detect tags like "P-101", "C-204" (pattern: letter prefix + dash + number).
8. **Entity extraction:** equipment, failure modes, personnel/roles, dates, regulatory references (via LLM structured extraction call).
9. **Relationship extraction:** LLM-assisted pass linking entities (e.g., "Pump P-101 → has_manual → Doc123").
10. **Embedding generation:** vector embedding per chunk (see stack, Section 13).
11. **Indexing:** write to vector DB + keyword index + upsert graph nodes/edges.

7.3 Unified Asset & Operations Brain

The graph layer connects Assets ↔ Manuals ↔ SOPs ↔ Incidents ↔ Maintenance history ↔ Failure modes ↔ Work orders ↔ Locations ↔ People/roles ↔ Inspection findings. Every document chunk, once entity-extracted, is linked to the graph node for the equipment tag it mentions — so a query about "P-101" pulls from every document type at once, not just one silo. This is the direct mechanism solving the "7–12 disconnected systems" problem from the brief.

7.4 Hybrid RAG System

Retrieval flow: query → query rewriting (expand abbreviations/tags) → parallel (a) vector search top-K, (b) BM25 keyword search top-K, (c) graph traversal from any detected asset tag → merge candidates → rerank (cross-encoder or LLM-scored) → top-N context → LLM generation with citation mapping.

- **Top-K strategy:** vector K=20, BM25 K=20, graph neighbors depth-2; merged and deduped before rerank to top 6–8 for the LLM context window.
- **Reranking:** cross-encoder (e.g., bge-reranker) or, for hackathon speed, an LLM relevance-scoring pass.
- **Context window management:** truncate/compress low-relevance chunks; hard cap context to fit model window with room for the answer.
- **Hallucination reduction:** system prompt forces "answer only from provided context; if not present, say so"; every claim must map to a retrieved chunk ID.
- **Confidence scoring:** derived from reranker score distribution + citation coverage (% of answer sentences with a matching source).
- **Fallback behavior:** if confidence below threshold, respond with best-effort answer explicitly labeled "low confidence — no strong source match" plus nearest related documents, instead of a fabricated definitive answer.

7.5 Industrial Knowledge Graph

Node types: Plant, Area, System, Equipment, Component, Document, SOP, Incident, WorkOrder, FailureMode, Person/Role, InspectionFinding.

Edge types: located_in, part_of, has_manual, has_sop, experienced_failure, performed_on, related_to, reported_by, assigned_to.

Example hierarchy: Plant → Area → System → Equipment → Component

```
(Equipment:P-101)-[:has_manual]->(Document:PumpManual_v3)
(Equipment:P-101)-[:experienced_failure]->(FailureMode:BearingWear)
(WorkOrder:WO-4521)-[:performed_on]->(Equipment:P-101)
(Incident:INC-089)-[:related_to]->(Equipment:P-101)
```

7.6 AI Assistant / Copilot

Capabilities: natural-language Q&A, troubleshooting guidance, maintenance-history lookup, SOP discovery, incident similarity search, RCA assistance, asset-specific filtered answers, cross-document reasoning, follow-up questions with conversation memory (session-scoped, last N turns).

Example queries (from brief): <cite index="0-1">"What is the shutdown procedure for Pump P-101?" / "Show previous failures similar to this vibration issue." / "What maintenance was performed on Compressor C-204?" / "Which SOP applies to this pressure anomaly?" / "What recurring failure patterns exist for this equipment?"</cite>

7.7 Explainability and Source Citations

Every answer includes: inline citation markers → document name + page number → an expandable evidence snippet (highlighted source text) → overall confidence score → a "Why this answer?" toggle showing the exact retrieved chunks and their scores.

7.8 Asset 360 View

Single page per equipment tag: header (name, tag, location, status), tabs for Manuals, SOPs, Maintenance History, Incidents, Work Orders, Known Failure Modes, Related Assets (graph neighbors), and an "AI Insights" panel summarizing current risk/gap signals for that asset.

7.9 Similar Incident Discovery

Vector similarity search over incident-report embeddings + symptom-text matching + failure-mode/asset-type filtering, surfaced as a ranked list with resolution notes so an engineer can compare past fixes.

7.10 Knowledge Gap Detection

Rule + LLM-assisted checks: equipment nodes with zero linked SOP/manual → flagged "undocumented asset"; documents unmodified beyond a staleness threshold → flagged "possibly outdated"; repeated incidents on the same asset without a resolved status → flagged "unresolved recurring issue"; low-confidence-answer queries logged as "knowledge gap candidates."

7.11 Analytics Dashboard

Most-searched assets, frequent operational issues, recurring failure patterns, knowledge gaps count, unresolved/low-confidence questions log, document freshness distribution, average AI confidence trend, active-user adoption metrics.

7.12 Admin Panel

User management, role management, document management (view/reprocess/delete), data source management, ingestion job monitor (status, failures, retry), model/prompt configuration, knowledge graph browser/editor, audit log viewer.

8. Functional Requirements Table

ID	Feature	Description	User	Priority	Accept Criteria
FR-001	Document upload	Upload PDF/DOCX/XLSX/image via UI	Knowledge Admin	P0	File appears in ingestion queue 2s of t
FR-002	OCR processing	Scanned pages converted to searchable text	Admin/Pipeline	P0	≥90% 'level C' accuracy clean
FR-003	Equipment tag extraction	Detect tags like P-101 in text	Pipeline	P0	≥85% 'on sys' test set
FR-004	Chunking & embedding	Semantic chunks embedded and indexed	Pipeline	P0	100% 'parsed' documents produced indexed chunk
FR-005	Hybrid retrieval	Vector + keyword + graph search merged	Backend/AI	P0	Retrieve relevant chunk 90% of benchmark queries
FR-006	Cited chat answer	Copilot answers with source citation	All users	P0	100% 'answers' include citations 'explicitly found'
FR-007	Asset 360 page	Unified per-equipment view	Maintenance/Reliability	P0	Page loads all linked entities given 1 <2s

ID	Feature	Description	User	Priority	Acceptance Criteria
FR-008	Similar incident search	Find past incidents like current one	Maintenance/Reliability	P1	Top-5 ranked similar score
FR-009	Knowledge graph explorer	Visual graph browsing	Reliability/Admin	P1	Graph renders hops from selected node
FR-010	Knowledge gap report	List undocumented/stale assets	Quality/Ops Manager	P1	Report assets with zero links to SOP/n
FR-011	Analytics dashboard	Usage & confidence metrics	Ops Manager	P1	Dashboard shows defined metrics updated daily
FR-012	RBAC	Role-based document/asset permissions	Enterprise Admin	P1	Unauthorized role blocked from restricted documents (403)
FR-013	Ingestion monitoring	View/retry failed jobs	Knowledge Admin	P1	Failed jobs show error messages and retry options
FR-014	Mobile-responsive UI	Usable on phone/tablet	Field Technician	P0	Core content usable on 375px viewport

ID	Feature	Description	User	Priority	Acceptance Criteria
FR-015	Compliance evidence export	Auto-generate audit packet	Quality Engineer	P2	Export PDF/C bundle linked for a regular query

9. Non-Functional Requirements

Category	Requirement	Target
Performance	Chat answer latency	p50 < 4s, p95 < 8s (hackathon scale)
Scalability	Documents supported	10k+ documents, horizontally scalable ingestion workers
Availability	Uptime (prod target)	99.5%
Reliability	Ingestion job success rate	≥95% on supported formats
Security	Data isolation	Per-organization tenant isolation (row-level or schema-level)
Privacy	PII handling	Redaction/flagging of detected personal data before storage
Explainability	Citation coverage	100% of generated answers cite ≥1 source or state uncertainty
Auditability	Action logging	All admin/ingestion/access actions logged with timestamp + actor
Accessibility	WCAG	AA contrast, keyboard-navigable chat UI
Maintainability	Modular services	Ingestion, retrieval, and graph services independently deployable
Observability	Monitoring	Structured logs + basic metrics (latency, error rate) per service

10. UX and UI Requirements

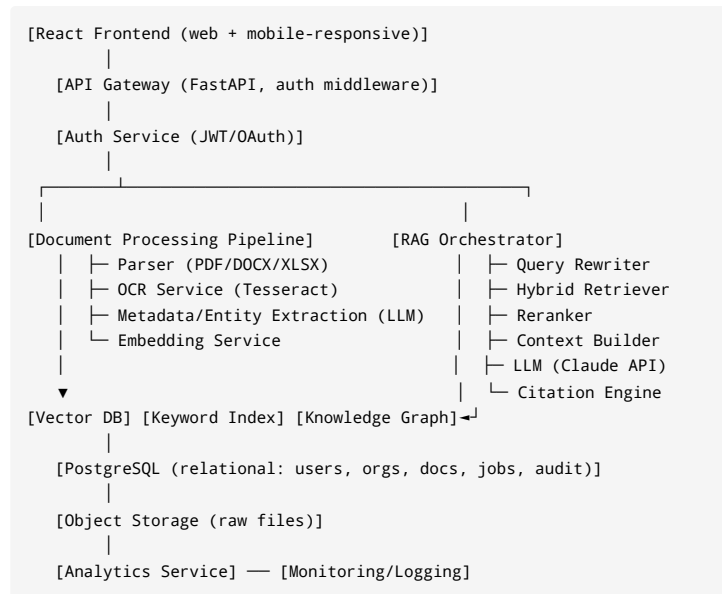
Page	Purpose	Key Components	Empty State
Login	Auth entry	SSO/email login, org selector	—
Dashboard	At-a-glance health	Metric cards, recent activity	"No data yet — upload documents"
AI Assistant	Core Q&A	Chat thread, citation panel, confidence badge	"Ask your first question"
Knowledge Explorer	Browse all knowledge	Filterable list (asset/type/date)	"No documents match filters"
Asset Explorer	Browse equipment	Searchable asset list/tree	"No assets ingested yet"
Asset 360	Per-equipment deep dive	Tabs (manuals/SOPs/history/incidents/AI insights)	"No linked documents for this asset — flagged as knowledge gap"
Document Library	Manage source docs	Table, upload button, status badges	"No documents uploaded"
Knowledge Graph Explorer	Visualize relationships	Interactive node/edge graph	"Graph builds as documents are ingested"

Page	Purpose	Key Components	Empty State
Incident Similarity	Compare incidents	Query box + ranked result cards	"No similar incidents found"
Analytics	Usage/quality metrics	Charts, tables	"Not enough data yet"
Admin Panel	Users/docs/jobs/audit	Tabs: Users, Docs, Jobs, Audit	Per-tab empty states

11. End-to-End User Flows

- 1. Upload a document:** Admin → Document Library → Upload → validation → ingestion queued → status updates (parsing → OCR → chunking → embedding → graph-linking → complete) → document appears searchable.
- 2. Ask an asset-specific question:** User → AI Assistant → types query with tag → query rewritten → hybrid retrieval → reranked → LLM answer with citations → user expands "Why this answer?".
- 3. Find an SOP:** User → asks "which SOP applies to X" → Copilot returns SOP title + section + link → user opens Document Library preview.
- 4. Investigate equipment failure:** Engineer → Asset 360 (P-101) → Maintenance History tab → sees prior work orders → clicks related Incident → views resolution notes.
- 5. Discover similar incidents:** Engineer → Incident Similarity page → describes symptom → ranked similar past incidents with resolutions.
- 6. View asset history:** User → Asset Explorer → selects asset → Asset 360 loads full timeline.
- 7. Trace answer citations:** User → clicks citation chip in chat → source document opens at cited page, highlighted snippet.
- 8. Admin monitors ingestion:** Admin → Admin Panel → Ingestion Jobs tab → sees failed job → views error → clicks Retry.

12. System Architecture



Data flow: Upload → object storage → pipeline workers write parsed chunks + entities to Postgres, vectors to vector DB, relationships to graph DB. Query → gateway → RAG orchestrator reads from vector DB + keyword index + graph → Claude API generates answer → citation engine maps sentences to chunk IDs stored in Postgres → response returned with citations → interaction logged to analytics/audit.

13. Recommended Technology Stack (Hackathon MVP)

Layer	Choice	Why	Alternatives	Trade-off
Frontend	React + Tailwind CSS	Fast to build, matches your existing MERN experience	Next.js	Next.js adds overhead not needed for a hackathon SI
Backend	FastAPI (Python)	Native fit with AI/ML pipeline, async support	Node.js	Node better i team is JS-or FastAPI wins RAG/OCR glu code

Layer	Choice	Why	Alternatives	Trade-off
AI orchestration	Custom lightweight orchestration + Claude API	Full control over hybrid retrieval logic for judging transparency	LangChain/LlamaIndex	Frameworks speed setup obscure the pipeline you must explain judges
LLM	Claude (Sonnet-tier via API)	Strong instruction-following for citation-constrained answers	OpenAI/Gemini/open-source	Open-source needs GPU hosting — no worth it in 72
Embeddings	BGE or E5 (open-source) via sentence-transformers	Free, local, no per-call cost during a hackathon	OpenAI embeddings	OpenAI simp but adds cost/depend
Vector DB	Qdrant (or ChromaDB for speed)	Easy local Docker setup, good hybrid-search support	Pinecone/Weaviate/pgvector	Pinecone requires account/cost pgvector simplest if already on Postgres
Keyword search	Postgres full-text search (or OpenSearch if time allows)	Zero extra infra	Elasticsearch/OpenSearch	Elastic better scale, overkill MVP
Knowledge graph	NetworkX (in-memory) for demo; Neo4j if team has bandwidth	NetworkX needs zero setup time	Neo4j	Neo4j is the "real" scalable choice but cc setup hours, don't have

Layer	Choice	Why	Alternatives	Trade-off
OCR	Tesseract	Free, offline, fast enough for demo scans	PaddleOCR/cloud OCR	PaddleOCR better accuracy on tables, mc setup
Storage	PostgreSQL + local/S3-compatible object storage	Single DB for relational + metadata	MongoDB	Postgres better for relational asset/graph-adjacent queries

14. Database Design (Core Schemas)

```

Users(id PK, org_id FK, name, email, role, created_at)
Organizations(id PK, name, created_at)
Assets(id PK, org_id FK, tag, name, type, location_id FK,
parent_asset_id FK)
Documents(id PK, org_id FK, title, type, storage_uri, hash, status,
uploaded_by FK, created_at)
DocumentVersions(id PK, document_id FK, version_no, storage_uri,
superseded_by FK, created_at)
Chunks(id PK, document_id FK, asset_id FK nullable, page_no,
section_title, text, embedding_id, created_at)
Incidents(id PK, org_id FK, asset_id FK, description, severity,
resolution, status, created_at)
WorkOrders(id PK, org_id FK, asset_id FK, description, performed_by,
performed_at, status)
MaintenanceRecords(id PK, asset_id FK, work_order_id FK, notes,
created_at)
Conversations(id PK, user_id FK, org_id FK, created_at)
Messages(id PK, conversation_id FK, role, content, confidence_score,
created_at)
Citations(id PK, message_id FK, chunk_id FK, page_no, snippet)
IngestionJobs(id PK, document_id FK, stage, status, error_message,
created_at, updated_at)
AuditLogs(id PK, org_id FK, user_id FK, action, target_type, target_id,
created_at)

```

Relationships: Assets.parent_asset_id → Assets.id (hierarchy);

Chunks.document_id → Documents.id; Citations.chunk_id → Chunks.id;

Incidents/WorkOrders.asset_id → Assets.id.

15. API Design (Representative Endpoints)

Method	Endpoint	Purpose
POST	/auth/login	Authenticate user, return JWT
POST	/documents/upload	Upload file, enqueue ingestion job
GET	/documents/{id}/status	Poll ingestion status
POST	/chat/query	Submit question, return cited answer
GET	/assets/{tag}/360	Get unified asset view
GET	/incidents/similar?text=	Semantic similar-incident search
GET	/graph/{node_id}/neighbors	Graph traversal from a node
GET	/analytics/dashboard	Aggregated usage/quality metrics
GET	/admin/jobs?status=failed	List failed ingestion jobs
POST	/admin/jobs/{id}/retry	Retry a failed job

Example – POST /chat/query

```
Request: { "conversation_id": "c123", "query": "What maintenance was
performed on Compressor C-204?" }
Response: {
  "answer": "Three work orders were logged on C-204 in 2025...",
  "confidence": 0.87,
  "citations": [
    {"document": "WO_Log_2025.xlsx", "page": null, "snippet": "C-204
bearing replacement, Mar 2025"}
  ]
}
```

16. AI/RAG Technical Design – Pipeline Pseudocode

```
def answer_query(query, org_id, conversation_history):
    rewritten = rewrite_query(query, conversation_history)      #
    expand_tags/abbreviations
    asset_tags = extract_asset_tags(rewritten)                  # regex
    + NER

    vector_hits = vector_search(rewritten, org_id, k=20)
```

```

keyword_hits = bm25_search(rewritten, org_id, k=20)
graph_hits = graph_traverse(asset_tags, depth=2) if asset_tags else
[]

candidates = dedupe(vector_hits + keyword_hits + graph_hits)
reranked = rerank(query, candidates, top_n=8)

context = build_context(reranked, max_tokens=6000)
prompt = construct_prompt(query, context,
citation_instructions=True)

response = llm_generate(prompt) #
Claude API call
citations = map_sentences_to_chunks(response, reranked)
confidence = score_confidence(citations, reranked)

if confidence < THRESHOLD:
    response = flag_low_confidence(response)

log_interaction(query, response, citations, confidence)
return {"answer": response, "citations": citations, "confidence":
confidence}

```

17. Chunking Strategy

Content type	Recommended strategy	Why
Manuals	Layout-aware + recursive (by heading/section)	Preserves procedural step order and section citation
SOPs	Layout-aware, small chunks (~200–300 tokens)	Steps must not be merged across procedures
Incident reports	Semantic chunking (paragraph-based)	Narrative text benefits from meaning-based boundaries
Tables (maintenance records)	Row-level parent-child chunking (row = child, table = parent context)	Preserves structured fields without flattening loss
Maintenance records	Fixed-size fallback if unstructured, else row-level	Mixed format tolerance

18. Security and Enterprise Requirements

RBAC (role → allowed actions/documents); organization isolation via `org_id` scoping on every query; document- and asset-level permission checks before retrieval; encryption at rest (DB/object storage) and in transit (TLS); audit logs for all reads/writes on sensitive documents; secure file upload (type/size validation, malware scan stub); prompt-injection defense (system prompt hardening + input sanitization, ignore instructions embedded inside retrieved documents); data leakage prevention (never let retrieved context from Org A appear in Org B's answers); PII detection/redaction pass during ingestion.

19. Evaluation Framework

- **Retrieval:** Precision@K, Recall@K, MRR, nDCG against a hand-labeled benchmark query set (20–30 questions with known correct source documents).
 - **Answer quality:** relevance and faithfulness scored via LLM-as-judge against retrieved context; citation accuracy = % of cited chunks that actually support the claim.
 - **Hallucination rate:** % of answer sentences with no supporting citation.
 - **Latency:** p50/p95 end-to-end response time.
 - **User satisfaction:** thumbs up/down on each answer, tracked in analytics.
 - **Test dataset strategy:** build a small synthetic-but-realistic corpus (Section 24) plus 20–30 benchmark Q&A pairs with ground-truth source documents, used consistently pre/post any pipeline change.
-

20. MVP Scope for Hackathon

P0 (Must Have): document upload + parsing (PDF/DOCX/XLSX/scanned image), OCR, chunking + embedding, equipment tag extraction, hybrid retrieval (vector+keyword), cited chat answers, Asset 360 basic view, mobile-responsive chat UI.

P1 (Should Have): knowledge graph (NetworkX) with graph-based retrieval boost, similar-incident discovery, knowledge-gap detection (undocumented assets), analytics dashboard, admin ingestion monitor, basic RBAC.

P2 (Nice to Have): compliance evidence export, Neo4j-based graph explorer UI, reranker model (vs. simple score fusion), multi-org support, offline/degraded-mode mobile handling.

21. 48–72 Hour Hackathon Execution Plan

Hours	Frontend	Backend	AI/RAG	Data	DevOps	Dem
0–6	Wireframes, repo scaffold	API scaffold, DB schema	Pick embedding/LLM, test Claude API	Build synthetic dataset	Repo + Docker setup	Outl dem scrij
6–12	Chat UI shell	Upload endpoint	Parsing/OCR pipeline	Load sample docs	CI basic pipeline	—
12–24	Asset 360 UI	Ingestion job queue	Chunking + embedding + vector DB	Tag/entity extraction pass	—	—
24–36	Knowledge Graph UI, Analytics	Chat query endpoint	Hybrid retrieval + reranking	Build graph nodes/edges	—	—
36–48	Polish, mobile responsiveness	Admin panel APIs	Citation engine, confidence scoring	Similar-incident data	Deploy staging	Rec rouç dem run-thro
48–60	Bug fixes, empty/error states	Bug fixes	Knowledge-gap detection	Finalize benchmark Q&A set	Final deploy	Full dem rehe
60–72	Final polish	Final polish	Final tuning	—	Freeze build	Rec final dem vide decl

22. Team Task Breakdown (4–6 members)

Role	Responsibilities	Deliverables	Dependencies
Frontend Lead	Chat UI, Asset 360, mobile responsiveness	React app	API contracts from backend
Backend Lead	API gateway, auth, DB, job queue	FastAPI service	DB schema

Role	Responsibilities	Deliverables	Dependencies
AI/RAG Engineer	Pipeline, retrieval, prompting, citations	RAG service	Embeddings/vector DB choice
Data/Graph Engineer	Synthetic dataset, entity/graph extraction	Seed data + graph	Parsed documents
DevOps/Integration	Docker, deployment, CI	Deployed staging URL	All services containerized
Presentation Lead	Deck, demo video, pitch narrative	Deck + video	Working demo flow

23. Demo Strategy (5–7 min)

1. Show the pain: fragmented docs across formats (10s).
2. Upload a manual + incident report live (30s).
3. Show graph auto-linking the new asset (20s).
4. Ask a hard cross-document question in chat (30s).
5. Show hybrid retrieval happening (brief architecture callout) (20s).
6. Deliver cited, evidence-backed answer (30s).
7. Click citation → source document opens (15s).
8. Similar-incident discovery for a fault (30s).
9. Asset 360 view walkthrough (30s).
10. Knowledge graph explorer (20s).
11. Close on business impact numbers from the brief (35% time lost, 18–22% downtime) and your measured time-to-answer improvement (30s).

24. Sample Demo Dataset (Synthetic)

- **Assets:** Pump P-101, Pump P-102, Compressor C-204, Motor M-301.
- **Manuals:** "Centrifugal Pump P-101 O&M Manual v3," "Compressor C-204 OEM Manual."
- **SOPs:** "SOP-014 Pump Startup," "SOP-022 Pressure Anomaly Response."
- **Maintenance records:** WO-4521 (P-101 bearing replacement, Mar 2025), WO-4599 (C-204 seal inspection, Jun 2025).
- **Incidents:** INC-089 (P-101 vibration spike, resolved via bearing replacement), INC-102 (C-204 pressure drop, unresolved).

- **Relationships:** P-101 → has_manual → Manual v3; P-101 → experienced_failure → BearingWear; WO-4521 → performed_on → P-101; INC-089 → related_to → P-101.

25. Risks and Mitigations

Risk	Mitigation
Hallucination	Strict "answer only from context" prompting + citation-coverage confidence gating
Poor OCR on scans	Pre-test with representative scan quality; fallback to manual text entry for demo docs
Bad chunking splits procedures	Layout-aware chunking by heading, tested on real SOP structure
Weak retrieval on rare terms	Hybrid vector+keyword so exact tag matches aren't lost to semantic search
Missing metadata	Synthetic dataset pre-tagged with equipment IDs to guarantee demo reliability
Knowledge graph complexity	NetworkX in-memory for hackathon; skip full Neo4j unless time allows
Slow latency	Cap context size, cache embeddings, pre-warm demo queries
Security risks	Basic RBAC + org scoping even in MVP, don't skip it entirely
Demo failure	Pre-record a backup demo video in addition to live demo

26. Success Metrics

- **Product KPIs:** time-to-answer vs. manual search (target: >70% reduction).
- **AI quality KPIs:** citation coverage ≥95%, hallucination rate <5% on benchmark set.
- **Operational KPIs:** % of assets with complete documentation linkage.
- **User adoption KPIs:** queries per session, repeat usage in testing.
- **Hackathon demo KPIs:** judges' scored criteria — Innovation, Business Impact, Technical Excellence, Scalability, UX (weights per Section 2).

27. Competitive Differentiation

- **vs. traditional enterprise search:** organizes around assets/equipment, not files; answers questions, doesn't just return document lists.
 - **vs. basic document chatbots:** cross-document, cross-system reasoning via the knowledge graph, not single-corpus Q&A.
 - **vs. standard vector-only RAG:** hybrid retrieval (vector + keyword + graph) catches exact equipment-tag matches that pure semantic search misses.
 - **vs. generic AI copilots:** citation-gated, confidence-scored, and mobile/field-usable by design for a safety-relevant domain.
-

28. Innovation Opportunities

- **GraphRAG** (graph-augmented retrieval) — *realistic for hackathon*, core to this MVP.
 - **Agentic RAG** (multi-step tool-using agent for RCA) — *partially realistic*: a simple RCA-assist chain is feasible; full autonomous agent is stretch.
 - **Multimodal industrial intelligence** (P&ID/drawing understanding via vision models) — *stretch/P2*, mention as roadmap.
 - **Predictive knowledge recommendations** — *stretch*, describe as post-hackathon roadmap.
 - **Failure pattern detection** — *realistic*, simple frequency-based pattern mining is achievable in 72h.
 - **Knowledge gap detection** — *realistic*, rule-based version is core P1.
 - **Asset digital thread** — *long-term vision*, not a 72h build.
 - **Human feedback loops** (thumbs up/down improving retrieval) — *realistic as a logging mechanism*, full learning loop is stretch.
-

29. Final Recommended Product Scope

- **Exact MVP feature list:** upload/parse/OCR → chunk/embed → hybrid retrieval → cited chat → Asset 360 → knowledge graph (NetworkX) → similar-incident search → knowledge-gap flagging → analytics dashboard → basic admin/RBAC.
- **Exact tech stack:** React + Tailwind, FastAPI, Claude API, BGE/E5 embeddings, Qdrant/ChromaDB, Postgres full-text search, NetworkX, Tesseract OCR, PostgreSQL + object storage.
- **Exact architecture:** as in Section 12.
- **Exact 3-day build plan:** as in Section 21.
- **Exact demo scenario:** as in Section 23.

- **What NOT to build:** full Neo4j deployment, multi-tenant enterprise SSO, live plant sensor/API integrations, custom-trained embedding/LLM models, full offline mobile mode.
- **Top 5 differentiators:** asset-centric graph linkage; hybrid retrieval; citation-gated answers; knowledge-gap detection; mobile-first field usability.
- **Top 5 judging points to emphasize:** measured time-to-answer improvement (Business Impact), hybrid retrieval architecture depth (Technical Excellence), graph-linked cross-document reasoning (Innovation), clean modular services (Scalability), cited/confidence-scored mobile UI (User Experience).

30. Final Implementation Blueprint

Frontend modules: Chat/, AssetExplorer/, Asset360/, KnowledgeGraph/, DocumentLibrary/, Analytics/, Admin/.

Backend modules: auth/, documents/, ingestion/, chat/, assets/, incidents/, graph/, analytics/, admin/.

AI services: parser_service, ocr_service, entity_extraction_service, embedding_service, retrieval_service, rerank_service, citation_service.

Databases: PostgreSQL (relational + metadata), Qdrant/ChromaDB (vectors), NetworkX in-process graph (serialized to Postgres/JSON for persistence).

Folder structure (backend):

```
/backend
/app
  /auth
  /documents
  /ingestion
  /chat
  /assets
  /graph
  /analytics
  /admin
  /core (config, db session)
/workers (async ingestion jobs)
/tests
```

Development sequence: DB schema → auth → upload/ingestion pipeline → embedding/vector DB → retrieval → chat endpoint → citation mapping → frontend chat UI → Asset 360 → graph layer → similar-incident + gap detection → analytics/admin → polish/deploy.

Deployment plan (hackathon): Docker Compose (Postgres, Qdrant/Chroma, FastAPI, React build) on a single cloud VM or Render/Railway-style PaaS; environment variables for Claude API key; staging URL frozen ~6 hours before submission for demo stability.